EXPERIMENT 4 – CI/CD PIPELINE AND PROMPT-BASED AUTOMATION

Last-minute practical exam notes | Essential files and commands

Aim: Configure a CI/CD pipeline to test and deploy scripts, update Agile task status based on success or failure, and generate a report that summarizes results and detects errors.

Platform: GitHub Actions. The lab screenshots demonstrate a Python application, pytest tests, a deployment script, a workflow report artifact, and GitHub Issues.

1. Create the project folder

```
mkdir cicd-automation
cd cicd-automation
git init
```

2. Create the Python application

```
nano app.py
```

Enter:

```
print("CI/CD Demo Application")
print("2 + 3 =", 2 + 3)
print("2 * 3 =", 2 * 3)
```

```
python3 app.py
```

3. Create test file

```
nano test_app.py
```

Enter:

```
def test_addition():
    assert 2 + 3 == 5

def test_multiplication():
    assert 2 * 3 == 6

def test_negative_numbers():
    assert -2 + -3 == -5
```

Install and run pytest:

```
sudo apt install python3-pip -y
pip3 install pytest
pytest
```

Expected result: **3 passed**.

4. Create requirements file

```
nano requirements.txt
```

Enter:

```
pytest
```

5. Create deployment script

```
nano deployment.sh
```

Enter:

```
#!/bin/bash
echo "Application tested and deployed successfully"
```

```
chmod +x deployment.sh
./deployment.sh
```

6. Create GitHub Actions workflow

Create the workflow folder and file:

```
mkdir -p .github/workflows
nano .github/workflows/ci.yml
```

Enter this workflow. It runs on every push and can also be started manually from GitHub Actions:

```
name: CI CD Automation
```

```
on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.10'

      - run: pip install -r requirements.txt
      - run: pytest

      - name: Deploy
        run: bash deployment.sh

      - name: Create report
        if: always()
        run: |
          echo "CI/CD Automation Report" > report.txt
          echo "Pipeline completed: ${{ job.status }}" >> report.txt

      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: CI-CD-Automation-Report
          path: report.txt
```

7. Create prompt-based report script

This simple local report script reads the pipeline/test log, detects errors, and writes a summary. It does not require an external AI API key.

```
nano generate_report.py
```

```
log = open("test_log.txt").read()

if "failed" in log.lower() or "error" in log.lower():
    status = "FAILURE"
    action = "Check the errors and fix the code."
else:
    status = "SUCCESS"
    action = "All tests passed. Continue deployment."

with open("automation_report.txt", "w") as f:
    f.write("CI/CD Automation Report\\n")
    f.write("Status: " + status + "\\n")
    f.write("Summary: " + action + "\\n")

print("Report generated successfully")
```

Create a sample log and run the script:

```
pytest > test_log.txt 2>&1
python3 generate_report.py
cat automation_report.txt
```

8. Commit and push to GitHub

Create an empty GitHub repository first, then connect and push (replace USERNAME):

```
git add .
git commit -m "Add CI CD automation"
git branch -M main
git remote add origin https://github.com/USERNAME/cicd-automation.git
git push -u origin main
```

Use your configured GitHub authentication (PAT, GitHub CLI, or SSH).

9. Check pipeline and update Agile task

1. Open the repository on GitHub.
2. Select **Actions** and open the workflow run.
3. Check whether the test and deployment steps succeeded.

4. Download the **CI-CD-Automation-Report** artifact.
5. Open **Issues** and update the related task status: Success → Done; Failure → In Progress (or To Do while pending fixes).

Note: The lab requires Agile status updates based on pipeline results; the practical steps above update the issue manually in GitHub. The source does not provide an automatic issue-status API script.

10. Essential verification

```
python3 app.py
pytest
./deployment.sh
python3 generate_report.py
git status
```

Remember for the exam

- **CI:** Continuous Integration – automatically builds/tests code after changes.
- **CD:** Continuous Delivery/Deployment – prepares or deploys tested code.
- **GitHub Actions:** runs automated workflows from YAML files.
- **pytest:** runs Python test cases.
- **Artifact:** file saved from a workflow run, such as the report.
- **Prompt-based automation:** analyzes execution results, detects errors, and produces an actionable summary.

Exam conclusion: A GitHub Actions CI/CD workflow was configured to test the Python application, run deployment steps, and generate a report artifact. The pipeline result was reviewed and the corresponding Agile task status was updated.